

软件质量管理

- 软件过程

- 概念

- 管理是什么？

- 三大关键要素：目标、状态、纠偏

- 管理视角的软件工程

- 试图复制其他场景的成功

- 软件过程

- 为了实现一个或多个事先定义的目标而建立起来的一组实践集合
 - 这组实践具备一定先后顺序，是一个整体
 - 广义软件过程

- 产品质量很大程度上取决于过程质量

- 技术、人员、狭义过程
 - 亦称软件开发方法、软件开发过程 eg. 净室方法，XP方法，SCRUM方法，敏捷软件方法

- 生命周期模型

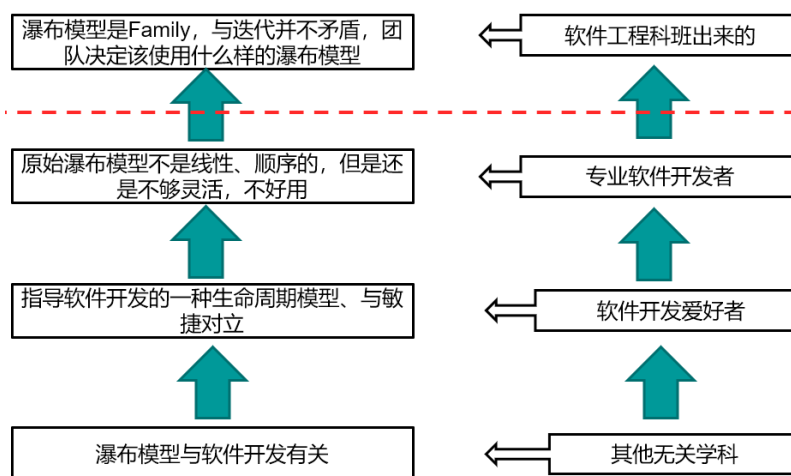
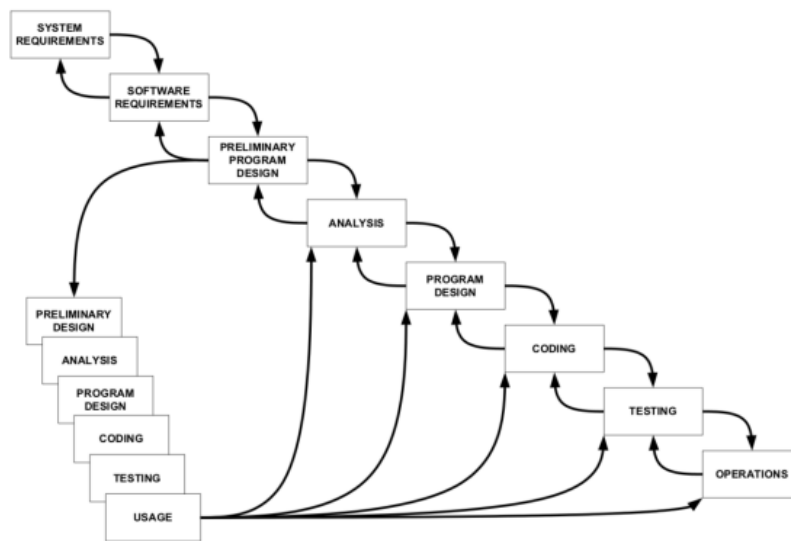
- 对一个软件过程的人为粗粒度划分
 - 是软件开发过程的主框架
 - 往往不包括技术实践

- 典型生命周期模型

- 迭代式模型

- 大型软件系统的开发过程也是一个逐步学习和交流的过程，软件系统的交付不是一次完成，而是通过多个迭代周期，逐步完成交付
 - 用户与开发人员的学习交流主动降低了需求变更的幅度与风险

- 瀑布模型



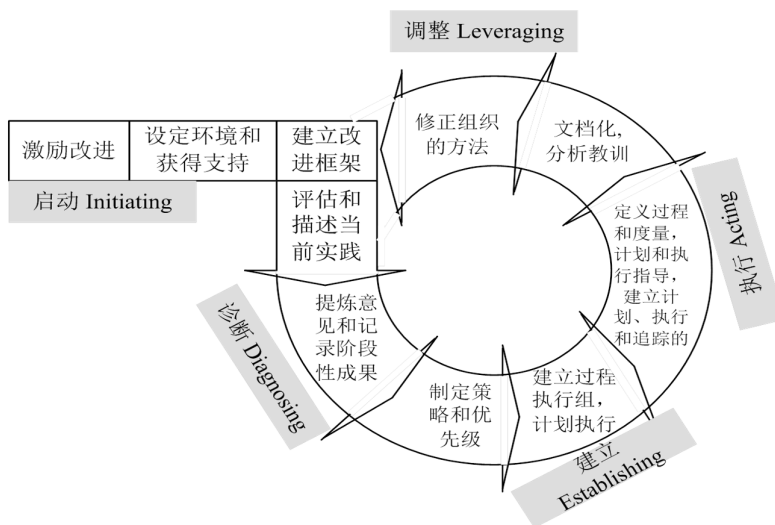
- 提出的本意是瀑布模型不是一个单一的生命周期模型，而是很多元素的融合，是系列的生命周期模型
- 软件项目应该结合实际情况选择合适过程元素的瀑布模型。基本原则是，项目面临困难和挑战越多，选择的模型应该越复杂。
- 软件团队应该根据软件开发实际情况调整其中的元素，而不是直接复读简单瀑布模型

• 软件过程管理

- PDCA和IDEAL



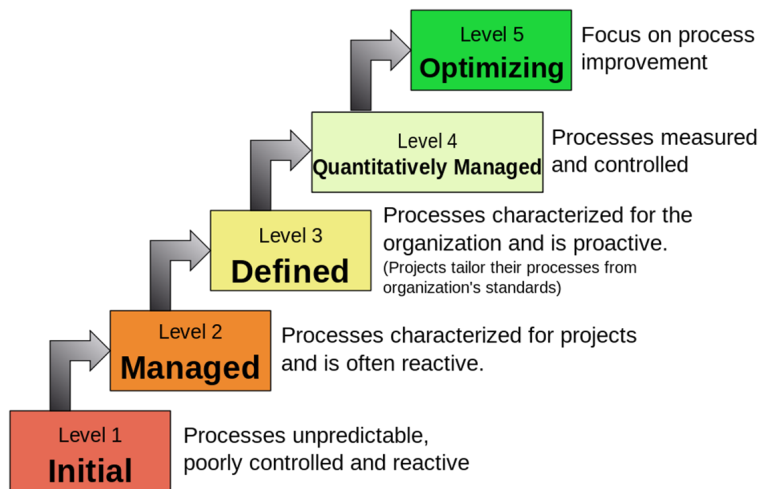
图 1.20 戴明的 PDCA 循环示意图



软件过程改进参考元模型：更加一般概念化，是参考模型的理论基础

- CMM/CMMI

Characteristics of the Maturity levels



一种软件过程管理参考模型，不能直接用于评估商业目标不同的公司间的能力强弱

1-3是定性的粗粒度的管理，4-5是定量的细粒度管理和改进

高等级相对低等级的本质差别是：通过数据和模型来管理和改进过程

- **Initial**: 初始状态，软件开发是一个黑盒，不可预测结果，盲目开发，救火文化
- **Managed**: 项目小组化管理，每个项目小组有一套可重复的方法，成功可被复制
- **Defined**: 公司层面定义标准化流程，降低经验教训在组织内的传播成本，各小组根据自己的特性裁剪规范
- **Quantitatively Managed**: 引入定量管理，能对未来的结果进行预测，通过让过程上下限预期尽可能接近，使得结果可预测
- **Optimizing**: 持续改进，通过引入新方法改进预测的期望值

• 软件工程演变的历史视角

• 软件危机

- 落后的软件生产方式无法满足迅速增长的计算机软件需求，导致软件开发与维护过程中出现一系列严重问题

• 软件工程

- 研究用工程化方法构建和维护有效、实用和高质量的软件的学科
- 管理视角+技术视角

• 三大阶段

• 软硬件一体化阶段

- 软件完全依附于硬件：Measure twice, cut once (Code Review)
支持硬件、功能单一、需求不需变更、硬件太贵
- 软件作坊：Code and Fix
功能简单、规模小、非专业人员涌入、质疑权威

• 软件成为独立产品：

硬件抽象层出现、功能强大、规模和复杂度剧增、PC出现带来市场，可变性和一致性要求

- 形式化方法：用数学方式描述并解决问题，扩展性、可用性存在不足

- 结构化程序设计和瀑布模型：重文档、慢节奏

- CMM/CMMI

- 网络化和服务化

功能更复杂、用户数量急剧增加、快速演化和需求不确定

- 迭代式
- 雪鸟会议和敏捷宣言
 - 个体和互动胜过流程和工具
 - 可以工作的软件胜过详尽的文档
 - 客户合作胜过合同谈判
 - 响应变化胜过遵循计划
 - 尽管右项有其价值，但我们更重视左项的价值，右项的基础上进一步发挥左项的价值

- XP, SCRUM, Kanban

Kanban：精益生产，每个生产周期短，目标精细化

- 开源软件开发方法

基于并行开发模式的软件开发组织与管理方式

- Linus定律：如果有足够多的beta测试者和合作开发者，几乎所有问题都会很快显现，然后自然有人会把它解决（已经不适用）
- 早发布、常发布、倾听用户反馈；把用户当成开发合作者对待；设计上的完美是无物可减
- 代码管理：严格的代码提交社区审核制度
- 演化：内部开源，众包

- 本质难题

本质难题是软件工程不断演变的驱动力

- 不可见性
- 复杂性
- 可变性
- 一致性

- 软件项目管理

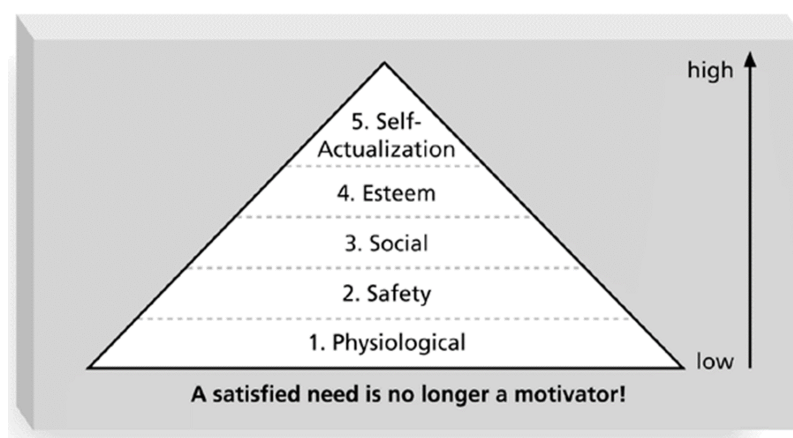
- 概念

- 三大目标
 - 成本、质量、工期

- 团队动力学

- 知识工作特点
 - 知识工作管理

- 关键规则：管理者无法管理工作者，知识工作者必须实现并且学会自我管理
- 自我管理
 - 有积极性
 - 能做出准确的估算和计划
 - 懂得协商承诺
 - 有效跟踪自己的计划
 - 持续地按计划交付高质量产物
- 领导者和特点
 - 知识工作者的管理需要的是领导者，而不是经理
 - 知识工作领导者特点
 - 诚实 honest
 - 博识 competent
 - 远见 visionary
 - 乐观 inspirational
- 不同的激励方式
 - 威逼、利诱、鼓励承诺
 - 交易型领导方式：威逼利诱
 - 转变型领导方式：鼓励承诺
 - 承诺在个人级别有很大差异
 - 团队承诺
 - 所有团队成员共同参与作出承诺
 - 团队依赖于每一位成员履行自己的承诺
 - 承诺自愿、公开、可行、向团队承诺
 - 交易型领导方式很少能产生成功的并且有创造性的团队
 - 马斯诺需求层次理论



用于指导选择激励手段，维持激励水平需要绩效反馈（度量进度、重做计划、里程碑）

- 五层（自底向上）

- 基本或生理需求
- 安全需求
- 爱与归属感
- 尊重
- 自我实现
- 激励来自为没有满足的需求而努力奋斗
- 低层次需求必须在高层次需求满足之前得到满足
- 满足高层次需求的途径比满足低层次需求的途径更为广泛

- **期望理论**

人们在下面的情况下能够受到激励并且产生大量成果：1.相信他们的努力很可能会产生成功的结果 2.相信自己会因为成功得到相应的回报

- $M = V * E$
- M：激发力量，调动一个人的积极性
- V：目标价值，指达到目标对于满足他个人需要的价值
- E：期望值，能够达到目标的概率

- **自主团队**

- **内外部环境**

- **内部环境**

- 自行定义项目的目标
 - 自行决定团队组成形式以及成员的角色
 - 自行决定项目的开发策略
 - 自行定义项目的开发过程
 - 自行制定项目的开发计划
 - 自行度量、管理和控制项目工作

- **外部环境**

- 项目启动阶段获得管理层的支持
 - 工作态度
 - 定期报告
 - 计划合理性
 - 高质量
 - 允许变更
 - 寻求帮助
 - 项目进展过程中获得管理层的支持
 - 严格遵循开发工作
 - 维护更新个人/团队计划

- 管理产品质量
- 跟踪进展报告
- 展现优异表现

- **TSP角色和职责**

- **TSP启动过程**

- 产品目标和业务目标
 - 角色分配和小组目标
 - 开发流程和策略选择
 - 整体计划
 - 质量计划
 - 个人计划和计划平衡
 - 风险评估
 - 准备汇报
 - 汇报计划
 - 总结

- **TSP角色**

- 项目组长
 - 激励
 - 主持会议
 - 汇报项目状态
 - 分配工作任务
 - 维护项目资料
 - 组织总结
 - 计划经理
 - 开发项目计划
 - 平衡计划
 - 跟踪项目进度
 - 参与总结
 - 开发经理
 - 制定开发策略
 - 估算
 - 需求规格说明
 - 高层设计
 - 设计规格说明
 - 软件产品

- 集成测试和系统测试
 - 用户支持文档
 - 参与总结
- 质量经理
 - 开发跟踪质量计划
 - 警示质量问题
 - 提交配置管理之前评审，消除质量问题
 - 组织协调评审
 - 参与总结
- 过程经理
 - 记录开发过程，支持过程改进
 - 建立维护开发标准
 - 记录维护会议记录
 - 参与总结
- 支持经理
 - 识别需要的各类工具和设施
 - 配置管理
 - 风险和问题跟踪
 - 复用策略应用
 - 参与总结
- 开发人员
- **SCRUM角色和职责**
 - SCRUM Master
 - 促进支持SCRUM
 - 帮助每个人理解SCRUM
 - 服务型领导
 - 帮助团队外的人了解如何与团队交互最有益
 - 改变团队内外互动方式来最大化团队创造价值
 - 服务于产品负责人
 - 确保 Scrum 团队中的每个人都尽可能地理解目标、范围和产品域
 - 找到有效管理产品待办列表的技巧
 - 帮助 Scrum 团队理解为何需要清晰且简明的产品待办列表项
 - 理解在经验主义的环境中的产品规划
 - 确保产品负责人懂得如何来安排产品待办列表使其达到最大化价值
 - 理解并实践敏捷性

- 当被请求或需要时，引导 Scrum 事件
- 服务于开发团队
 - 作为教练在自组织和跨职能方面给予开发团队以指导
 - 帮助开发团队创造高价值的产品
 - 移除开发团队工作进展中的障碍
 - 按被请求或需要时，引导 Scrum 事件
 - 在 Scrum 还未完全采纳和理解的组织环境中，作为教练指导开发团队
- 服务于组织
 - 带领并作为教练指导组织采纳 Scrum
 - 在组织范围内规划 Scrum 的实施
 - 帮助员工和利益攸关者理解并实施 Scrum 和经验导向的产品开发
 - 引发能够提升 Scrum 团队生产率的改变
 - 与其他 Scrum Master 一起工作，增强组织中 Scrum 应用的有效性
- 产品负责人
 - 将开发团队开发产品价值最大化
 - 管理产品待办列表
 - 清晰表述
 - 排序
 - 优化价值
 - 确保产品待办列表清晰可见，开发团队足够了解
- 开发团队
 - 每个Sprint结束时交付潜在可发布并且完成的产品增量
 - 团队特点：
 - 自组织
 - 跨职能
 - 平级
 - 无子团队
 - 责任均摊

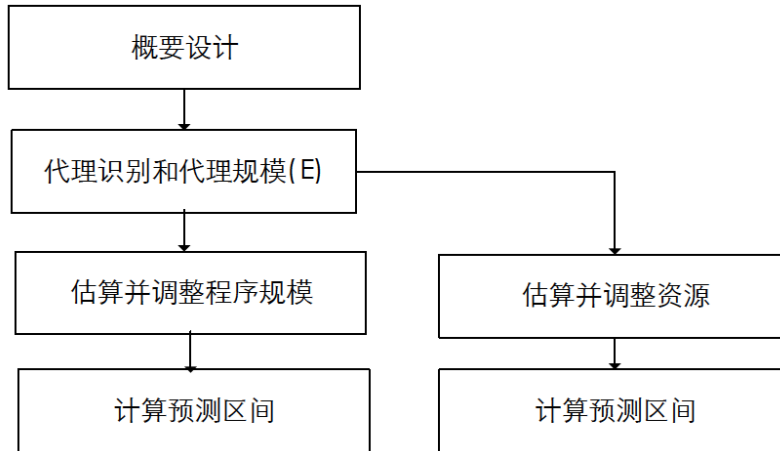
• 估算和计划

• 估算要点

- 估算目的：给各类计划提供决策依据
- 抽象的估算：混合了为开发特性所要付出的努力、开发复杂度、个中风险以及类似东西
- 相对的估算：设定标准之后，考虑其他特性与标准之间的相对大小关系，不是绝对数字

- 估算要点：
 - 尽可能划分详细
 - 尽量依赖数据
 - 目标是建立对结果的信心
 - 估算的重点是相关干系人达成一致共识的过程，而非结果

• PROBE估算方法



精确度量 and 早期规划之间的桥梁

• 相对大小矩阵

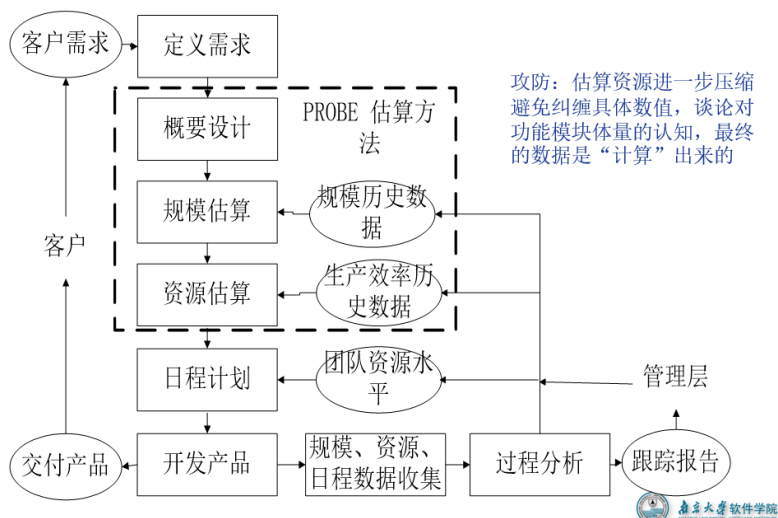
类型	VS	S	M	L	VL
Calculation	2.34	5.13	11.25	24.66	54.04
Data	2.60	4.79	8.84	16.31	30.09
I/O	9.01	12.06	16.15	21.62	28.93
Logic	7.55	10.98	15.98	23.25	33.83
Set-up	3.88	5.04	6.56	8.53	11.09
Text	3.75	8.00	17.07	36.41	77.66

不要直接估算结果，而是先估算模块特征

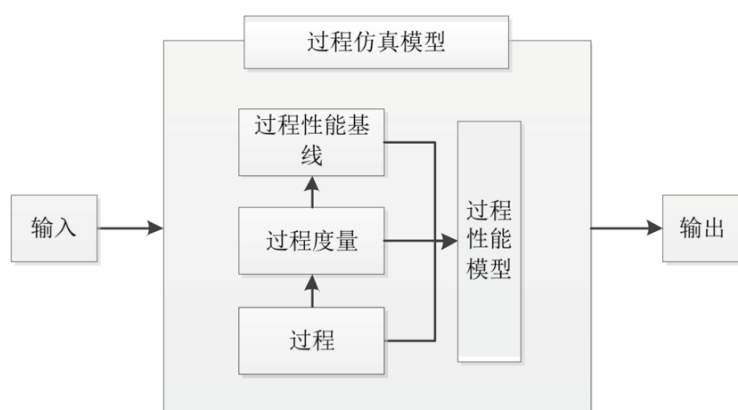
• SCRUM故事点

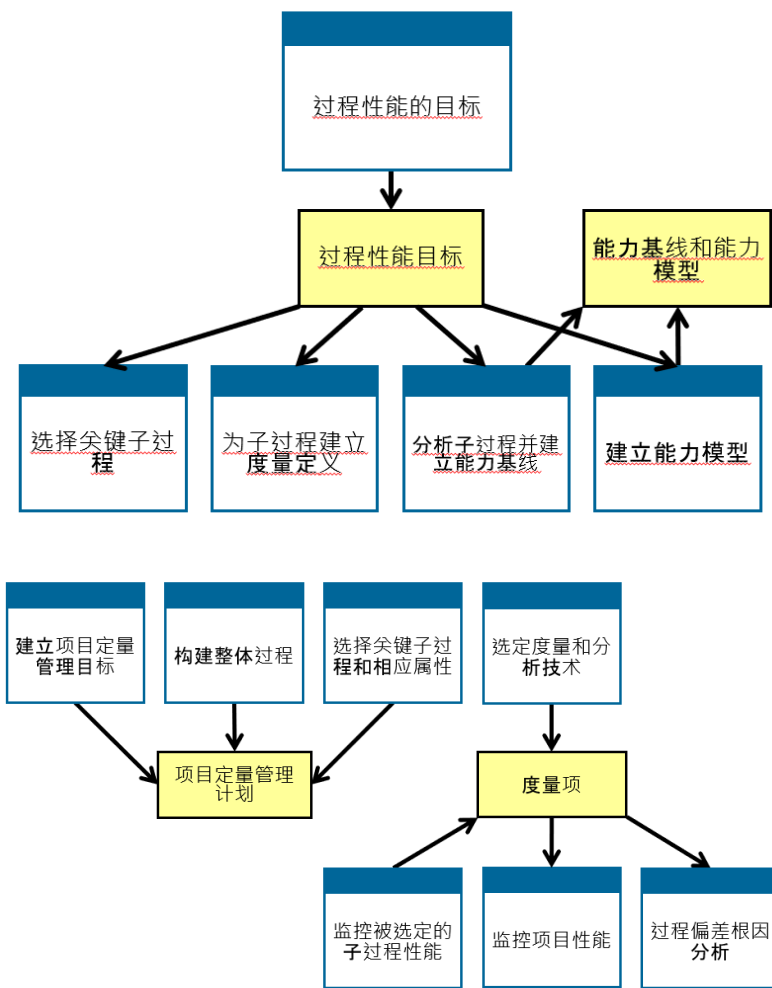
度量实现一个故事需要付出的工作量，利用斐波那契数列来为每个用户故事分配故事点数值。

• 通用计划框架



- 以下步骤必须人为的干预
 - 定义需求
 - 概要设计：划分由人为开始，规模划分好之后估算是自动产生的
 - 日程计划
- 这会带来什么的好处？答：比较容易扛住别人的质疑。
 - 攻击点：资源和时间是否被高估了
 - 解决：估算没有代码行，PROBE只有功能点，以大中小划分。
- 各类计划
 - 日程计划
 - 任务清单
 - 资源清单
 - 日程计划
 - 质量计划
 - 确定需要开展的质量保证活动，这些活动开展的程度
 - 包括个人评审、团队评审、单元测试、集成测试、系统测试以及验收测试
 - 风险计划
 - 在风险发生前，识别出潜在的问题，以便在项目生命周期中规划和实施风险管理活动，消除潜在问题对项目产生的负面影响
 - 风险识别：风险参数包括 Possibility 可能性，Impact 影响程度，Threshold 阈值
 - 风险应对
 - 风险转嫁：放弃部分利益，将部分风险转嫁到其他的团队或组织，例如外包
 - 风险解决：采取有效措施使风险来源不再存在，例如引入专家
 - 风险缓解：容忍风险的存在，采取措施监控风险，高风险或无法接受风险需要应急预案
- 定量管理计划（自顶向下）





- 过程模型

根据过程的逻辑关系构建相应的数学模型，描述子过程性能基线和整体过程有意义的性能输出之间的关系

- 过程能力基线

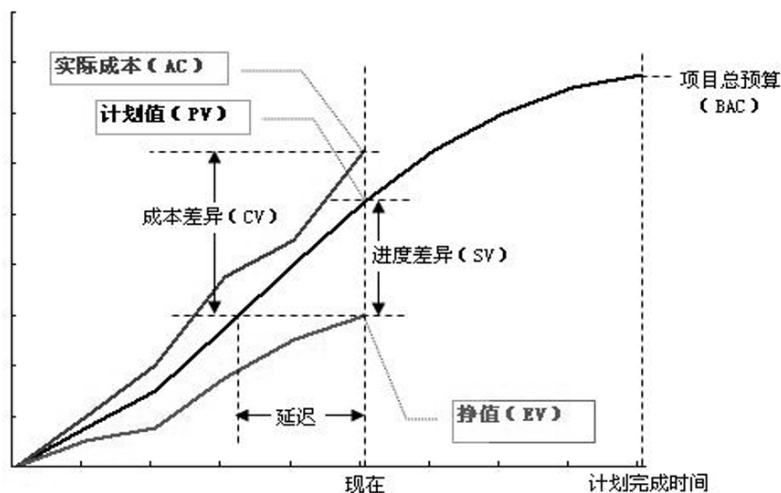
过程性能是遵循某个特定过程之后产生结果的量化描述，包括过程度量和产物度量，过程性能基线则是上述过程性能的定量化刻画，包括均值和范围

- 过程组合

关键子过程性能目标与整体过程性能目标

- 跟踪

- 挣值管理体系



客观度量项目进度的一种项目管理方法

PV = 计划值

EV = 挣值

SV = 进度差异 = EV - PV (中级)

SPI = 进度差异系数 = EV / PV (中级)

AC = 实际成本 (高级)

CV = 成本差异 = EV - AC (高级)

CPI = 成本差异系数 = EV / AC (高级)

EAC = 预计完成成本 = $AC + (BAC - EV) / CPI = BAC / CPI$

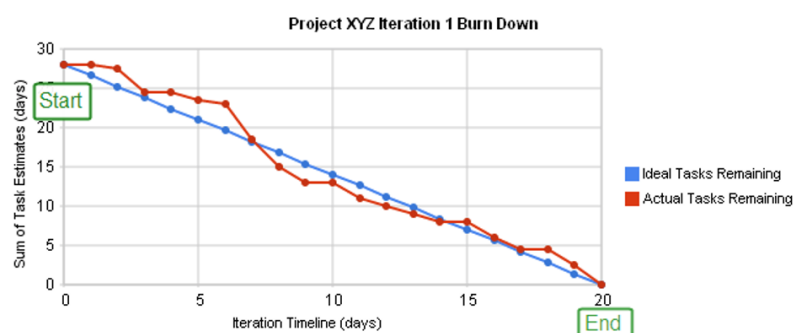
● 简单实现：仅仅关注进度信息

- 在实现时，首先需要建立WBS，定义工作范围；其次为WBS中每一项工作定义一个价值（PV）；最后按照一定的规则将某一数值赋给已经完成的工作或者正在进行的工作
- 0-100 规则，只有当某项任务完成时，该任务的 PV 值将转化成 EV 值
- 50-50 规则，只需要开始某项任务，即可以赋原PV值的50%作为EV值，完成时，再加上另外的50%
- 实际完成的工作所需成本AC不对EV值产生任何影响

● 中级实现：加入日程偏差计算

● 高级实现：考察项目的实际成本

● 燃尽图



EVM简单实现的变形

● 为何适应软件项目

- 适用于软件过程管理，一般不适用于软件质量管理

- 需要量化的管理机制
- 完全依赖于项目的准确估算（大家达成共识，出问题一起担）
- 很好适应需求的变更
- 定量管理的跟踪（自底向上）

$$Z = a \cdot Y_1 + b \cdot Y_2 + c$$

$$Y_1 = d \cdot X_1 + e \quad Y_2 = f \cdot X_2 + g$$

Cor Review Rate

TODO：只有听他上课的人才知道

- 过程性能

遵循某个特定过程之后产生结果的量化描述，包括过程度量 X_i （如时间、缺陷消除效率、工时等）和产物度量 Y_i （例如缺陷密度，相应时间等）

• 软件质量管理

- 概念

- 软件质量

与软件产品满足规定的和隐含的需求能力有关的特征或者特性的全体

- 内外两部分的特性

- 内部：不直接面向用户
- 外部：面向软件产品的最终用户

- 不同视角

- 软件质量为软件产品可以改变世界，使世界更加美好的程度（用户满意度是最为重要的判断标准）
- 软件质量是对人的价值，强调质量的主观性（对同一款软件而言，不同的用户对其质量有不同的体验）

- 质量管理的挑战：三要素

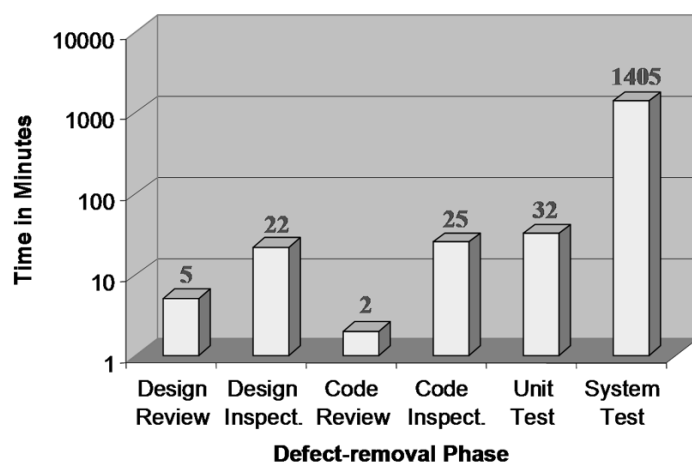
- 成本、质量、工期（日程）

- 面向用户的质量观

- 定义质量为满足用户需求的程度，需要明确：
 - 用户是谁
 - 用户需求的优先级是什么

- 用户的优先级对软件产品的开发过程产生什么样的影响
- 怎么度量这种质量观下的质量水平

• 质量管理策略和背后逻辑



尽可能在上游解决，不要拖到系统级；但不意味着系统测试就不需要



- 缺陷管理替代质量管理：高质量产品意味着组成软件产品的各个组件基本无缺陷
- PSP质量策略：各个组件的高质量是通过高质量评审实现的

• 评审

• 个人评审

- 关键控制因素：对评审对象的整体把握，评审人员的注意力（打印）
- 时机选择：编译前
 - 评审速度一定，先评审再编译能够节省更多时间
 - 编译器会遗漏一部分缺陷，仍然需要通过评审消除，消除后还需要编译
 - 编译中发现的错误过多意味着隐藏着更多缺陷
 - 编译前评审是自我学习的好机会
 - 编译过程中没有缺陷会带来极大的成就感
 - 影响个人心态，先编译会使得评审心态放松，导致评审质量低

• 小组评审

- 过程质量控制方式
 - 评审九宫图（评审速度 Review Rate 与缺陷密度，划分九个区域）
 - **纵轴（Review Rate）**：表示代码审查的频率或深度，分为低、中、高
 - **横轴（Defect Density）**：表示每单位代码中的缺陷数量，同样分为低、中、高
 - 两个人评审同一份代码
 - Catch-Re-Catch
 - 两个人的情况

假设评审人员 A 和 B 分别发现了 a 个缺陷和 b 个缺陷，其中 c 个缺陷两人同时发现。利用上述思想，选择 $a - c$ 和 $b - c$ 中较大值，如果相等则可以任选一值。

假设 $a - c$ 是选定的值，那么就可以把 a 当成上述第一网被标记的鱼， c 是第二网中被标记的鱼。简单计算就可以估算出评审对象经过小组评审之后，还遗留 $a \times \frac{b}{c} - (a + b - c)$ 个缺陷。

假设缺陷总数为 Z ，那么由 $\frac{a}{Z} = \frac{c}{b}$ 可以推出 $Z = \frac{a \times b}{c}$

- 遗留缺陷数 $a \times \frac{b}{c} - (a + b - c)$
- 缺陷总数 $Z = \frac{a \times b}{c}$
- 多个人的情况

我们采取了一个简化的计算方法。即选择某个独立发现缺陷最多的评审员作为A，而其他所有参与人员的整体作为B。那么我们仍然可以用上述相同的方式来估算小组评审之后评审对象中遗留的缺陷数。

• 质量控制指标

• Yield

用以度量每个阶段在消除缺陷方面的效率，用于制定计划时的预测

- $\text{Phase Yield} = 100 * (\text{某阶段发现的缺陷个数}) / (\text{某阶段注入的缺陷个数} + \text{进入该阶段前遗留的缺陷个数})$
- $\text{Process Yield} = 100 * (\text{第一次编译前发现的缺陷个数}) / (\text{第一次编译前注入的缺陷个数})$

• A/FR

用以衡量计划中评审时间的占比，期望值 2，用于制定计划时的预测

- $A/FR = \text{PSP质检成本} / \text{PSP失效成本}$
- PSP质检成本：设计评审时间与代码评审时间之和
- PSP失效成本：编译时间和单元测试时间之和

• PQI

反映设计过程中质量保证活动和最终缺陷数的某种关系，用于预测或者度量，期望 0.4

- 设计质量： $\min\{\frac{\text{设计时间}}{\text{编码时间}}, 1\}$
- 设计评审质量： $\min\{2 \times \frac{\text{设计评审时间}}{\text{设计时间}}, 1\}$
- 代码评审质量： $\min\{2 \times \frac{\text{代码评审时间}}{\text{编码时间}}, 1\}$
- 代码质量： $\min\{\frac{20}{\text{编译缺陷密度} + 10}, 1\}$ 代码编译缺陷密度小于 10 个/千行
- 程序质量： $\min\{\frac{10}{\text{单元测试缺陷密度} + 5}, 1\}$ 代码单元测试缺陷密度应当小于 5 个/千行

• DRL

度量不同缺陷消除手段消除缺陷的效率，只能度量不能预测

- 以某个测试阶段（一般为单元测试）每小时发现的缺陷数为基础，DRL是其他阶段（上游阶段）每小时发现缺陷数与该基础的比值

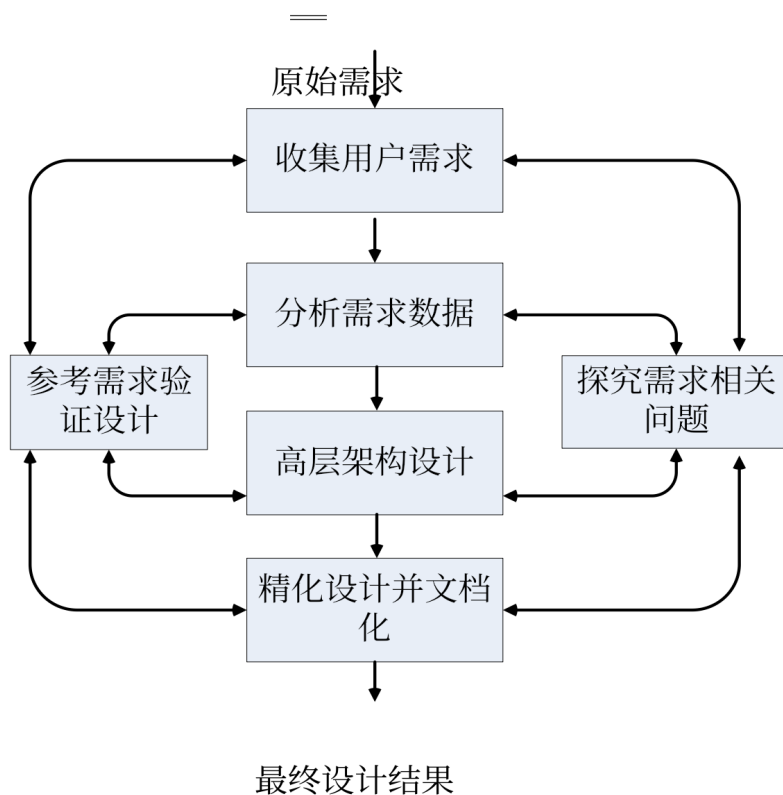
• 质量路径

Quality Journey

• 八个步骤

- 各种测试
- 进入测试之前的产物质量提升

- 评审过程度量 and 稳定
 - 质量意识和主人翁态度
 - 个体review的度量和稳定
 - 诉诸设计
 - 缺陷预防
 - 用户质量观——其他质量属性
- 设计



- 为什么要设计
 - 低劣的设计是导致在软件开发中返工、不易维护以及用户不满的主要原因。
 - 充分设计可以显著减少最终程序的规模，提升质量。
 - 设计本身也是一种排错的过程。
- 设计模板

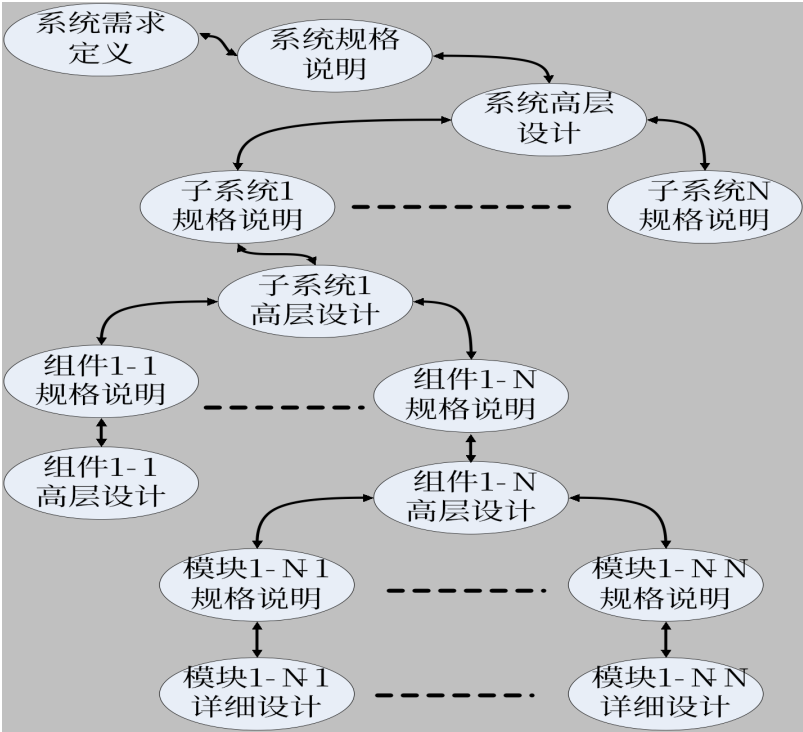
	动态信息	静态信息
外部信息	OST/FST	FST
内部信息	SST	LST

	动态信息	静态信息
外部信息	交互信息（服务、消息等）	功能（继承、类结构等）
内部信息	行为信息（状态机	结构信息（属性、业务逻辑等）

	动态信息	静态信息
外部信息	用例图、时序图	类图
内部信息	状态机图	？

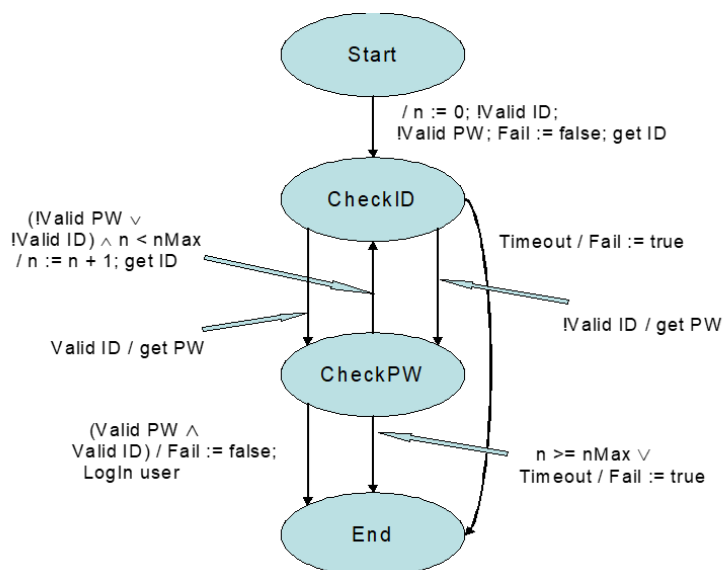
- OST 操作规格模板（运行时如何操作）
 - 系统与外界的交互，可用于定义测试场景和测试用例
- FST 功能规格模板（静态描述具备哪些功能）
 - 系统对外接口，包括类的继承关系、可见属性和方法等，消除二义性很重要，尽可能用形式化符号描述
- SST 状态规格模板（运行时存在哪些状态）
 - 精确定义程序的所有状态，状态转换和伴随的动作
- LST 逻辑规格模板（静态描述工作逻辑关系）
 - 精确描述内部逻辑，消除二义性很重要，形式化符号描述

设计层次



设计评审

- 状态机



- 检验状态机，消除死循环和陷阱
- 检查状态转换（使用真值表），验证完整性和正交性（同一状态的同一输入对应同一结果）
- 评价状态机，检验是否体现设计意图

- 符号化执行

适合验证复杂算法，不适合有复杂逻辑的情况，纯手工验证容易引入人为错误

- 识别伪代码程序中的关键变量
- 用代数符号表示这些变量
- 分析伪代码程序的行为

- 执行表，跟踪表

跟踪表添加了符号化用例和定义并优化用例组合，能够更一般化地验证程序行为

- 识别伪代码程序中的关键变量
- 构建表格，左侧填入主要程序步骤，右侧填入关键变量
- 初始化选定变量（具体数值）
- 跟踪关键变量的变化情况

- 正确性证明

形式化方法推理验证

- 分析识别用例
- 正确性检验地标准问题逐项验证
- 跟踪表等辅助验证

- 工程技术

- 需求

- 客户需求：问题
 - 产品需求：解决方案

- 产品经理？：获取客户需求，转化为产品需求

• 设计

- 自顶向下

• 考虑点

- 团队智慧两大挑战
 - 确定整体架构之前很难进行分工
 - 鼓励团队成员在讨论和评审会议中的参与程度
- 设计标准
 - 命名规范
 - 接口标准
 - 系统出错信息
 - 设计表示标准
- 复用性支持
 - 复用接口标准
 - 复用文档标准
 - 复用质量保证机制
- 可测试性支持
 - 尽可能减少测试代码的数量
 - 制作合理的测试计划
- 可用性支持
 - 设计阶段就开始考虑
 - 针对每一个关键功能都定义操作概念和场景
 - 分析操作场景确保使用者满意
 - 邀请最终用户参与场景评审

• 实现

• 评审

- 自底向上实现，先对底层模块评审，确保其质量，在坚实的底层支持之上再开发高层实现
- 有利于复用策略的应用

• 复用

- 编码注释（提醒其功能，调用方式和异常信息）
- 站立会议（识别可复用组件）

• 可测试性

- 实现计划与测试计划一致，避免集成测试时部分模块尚未实现

• 集成

- 覆盖范围

- 基本策略

- 大爆炸

- 所有组件一次集成
 - 模块质量高时效率最高，测试用例数量少
 - 难以定位缺陷位置，消耗很多测试时间

- 逐一添加

- CI/CD 属于该策略

- 一次添加一个组件
 - 容易定位缺陷位置
 - 需要非常多测试用例，测试时间成本高

- 集簇集成

- 优先集成相似、关联组件形成可工作组件，再不断向上集成
 - 尽早获得可工作组件，有利于其他测试工作的开展
 - 过于关注个别组件，缺乏整体观，不能尽早发现系统层面的缺陷

- 扁平化集成

- 自顶向下集成，不断替换桩
 - 可以尽早发现系统层面的缺陷
 - 需要大量桩程序，往往不能覆盖系统应该处理的多种状态

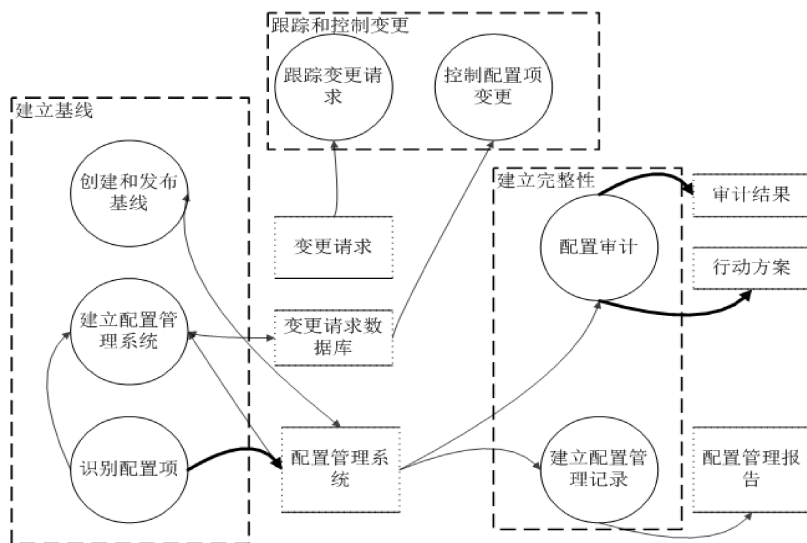
- V&V

验证和确认的目的不同，前者关注的是产品需求是否得到体现、从这个产品需求（即解决方案）设计出来，一直到解决方案的开发完成，整个过程当中有没有偏、解决方案有没有被正确地实现出来；后者更多关注的是你的这个客户需求有没有得到满足，简单来说就是客户期望解决的问题，有没有被解决。

- **Verification 验证**：对应产品需求
 - **Validation 确认**：对应客户需求

- 其他

- 配置管理



目的是建立与维护工作产品的完整性

• 配置项

- 配置管理中作为单独实体进行管理和控制的工作产品集合

• 基线

- 配置项持续演进的稳定基础
- 发布一个基线包括该基线所有的配置项以及这些配置项的最新变更
- 基线是接下来工作的基础，经典的发布基线包括需求分析之后，设计完成之后，单元测试之后以及最终产品发布

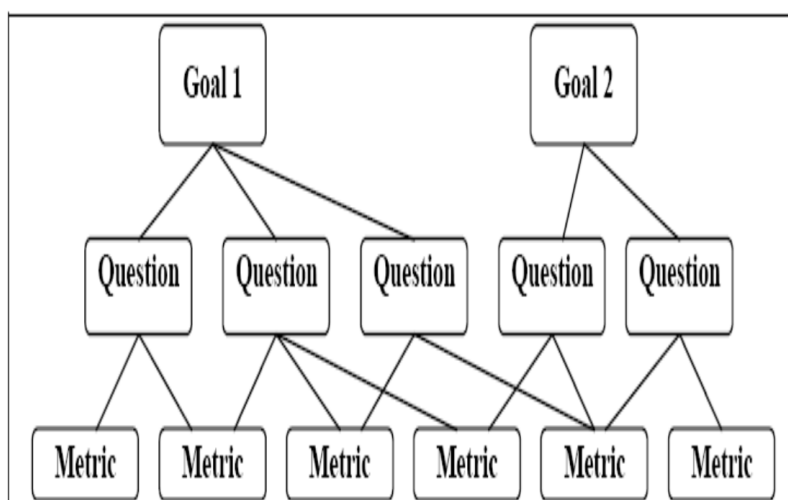
• 配置管理的活动

- 识别和记录配置项的物理特性和功能特性
- 控制上述特性的变更
- 记录和报告变更过程和相应的配置项状态
- 验证配置项是否与需求一致

• 度量和分析

支持项目管理

• GQM度量体系



一种应用非常广泛的建立软件度量体系的方法，从管理的目标出发，将目标归纳、分解为度量的指标，并把这些指标提炼为可以测量的值

- 概念层（目标）

目标是为某个特定的对象而定义的。这里的对象是指软件产品、软件过程以及相关的资源等。定义的目标基于不同原因和不同质量模型,也要参考不同的角色视图与特定的环境。

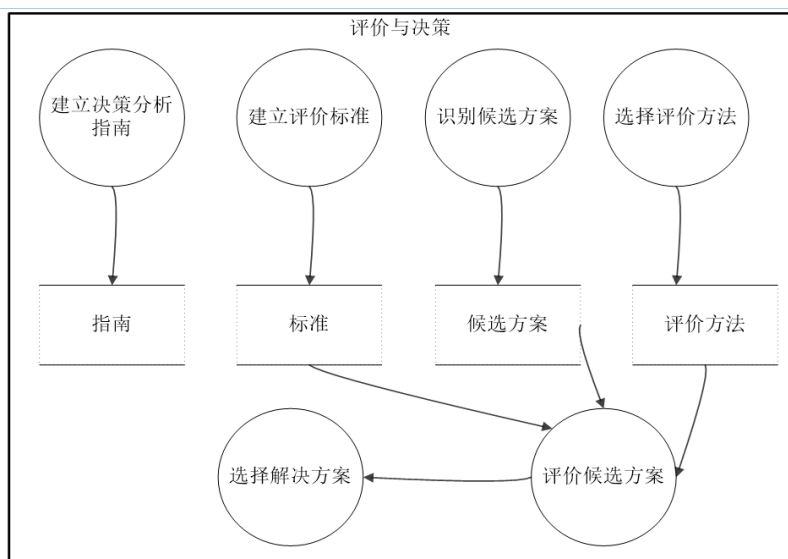
- 操作层（问题）

基于一定的刻画上述目标是否达成或者目标达成的进展情况的模型，使用一系列的问题来定义所研究的对象,然后得出评价或评估特定目标达成进展情况。所选择的问题应当尽量体现质量相关的话题。

- 量化层（度量）

试图以量化的方式回答上述操作层识别出来的问题。

- 决策分析与解决方案



- 招投标

- 投标不是决策分析
- 招标才是

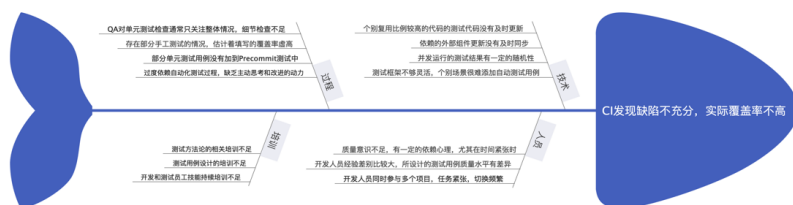
- 根因分析与解决方案

避免类似错误反复发生，包括识别选定问题、根因分析、建立改进的行动方案、实施改进并评估效果

- 2-8法则

- 20%的因素影响80%的结果
- “关键的少数”影响更大

- 鱼骨图



- 技术角度
- 人员角度

- 培训角度
- 过程角度